

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №7

«Безопасность браузера и анализ сетевого трафика»

Болорболд Аригуун

Группа: P3411

Выполнение

Настройка браузера

Для выполнения данной работы был выбран браузер Mozilla Firefox. Первым этапом стало изучение настраиваемых параметров безопасности, как показано на рисунке 1.

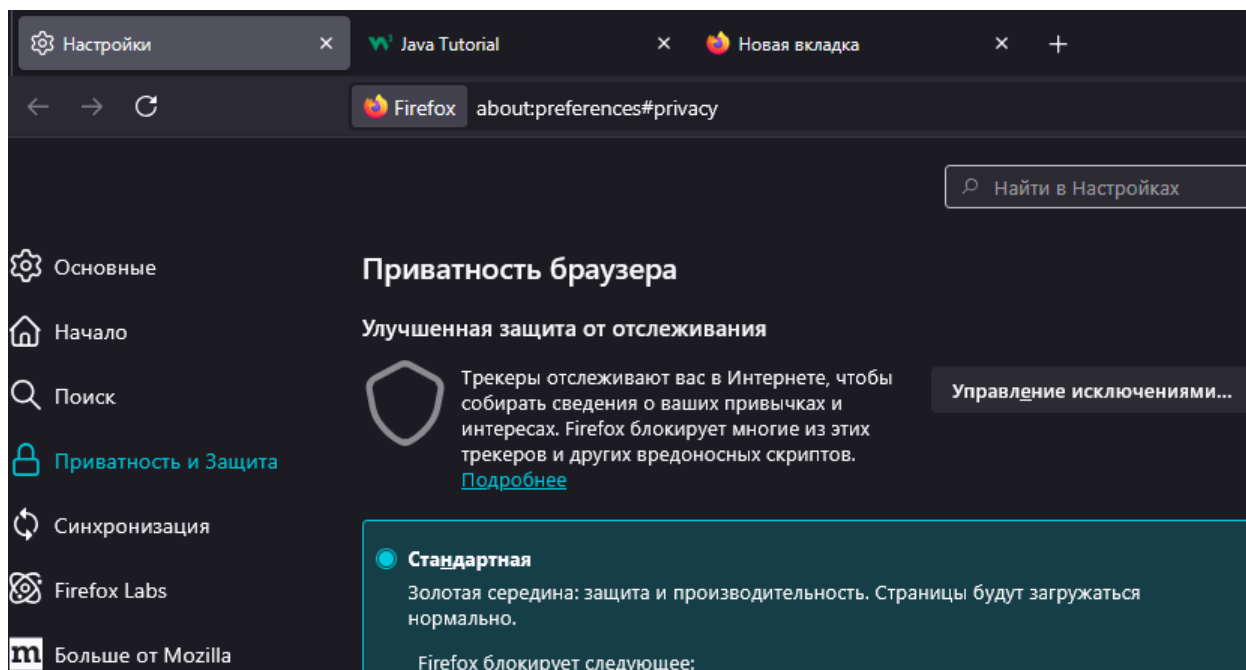


Рисунок 1 — Настройки приватности браузера

Сначала настроим файлы cookie. В отличие от других браузеров, как например Google Chrome, здесь не так уж много легкодоступных настроек. Для моего случая в учебной цели это хорошо, но на практике это не обязательно так.

Firefox предоставляет 3 выбора. Первый (рисунок 2) — *стандартная*:

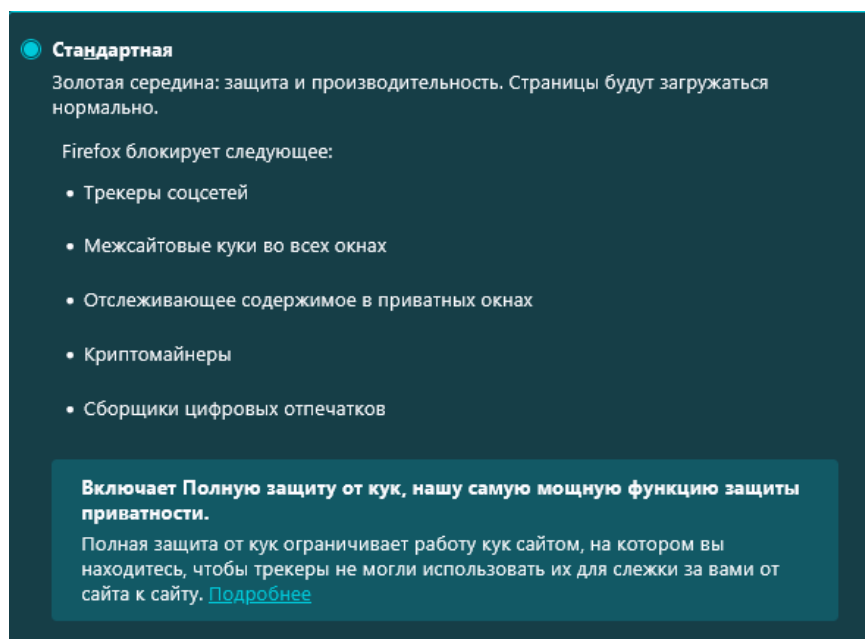


Рисунок 2 — Уровень настройки «Стандартная»

Вторая, *строгая* (рисунок 3), как понятно по названию, имеет более жесткие ограничения в пользу безопасности пользователя — даже если они являются в ущерб функциональности некоторых веб-сайтов.

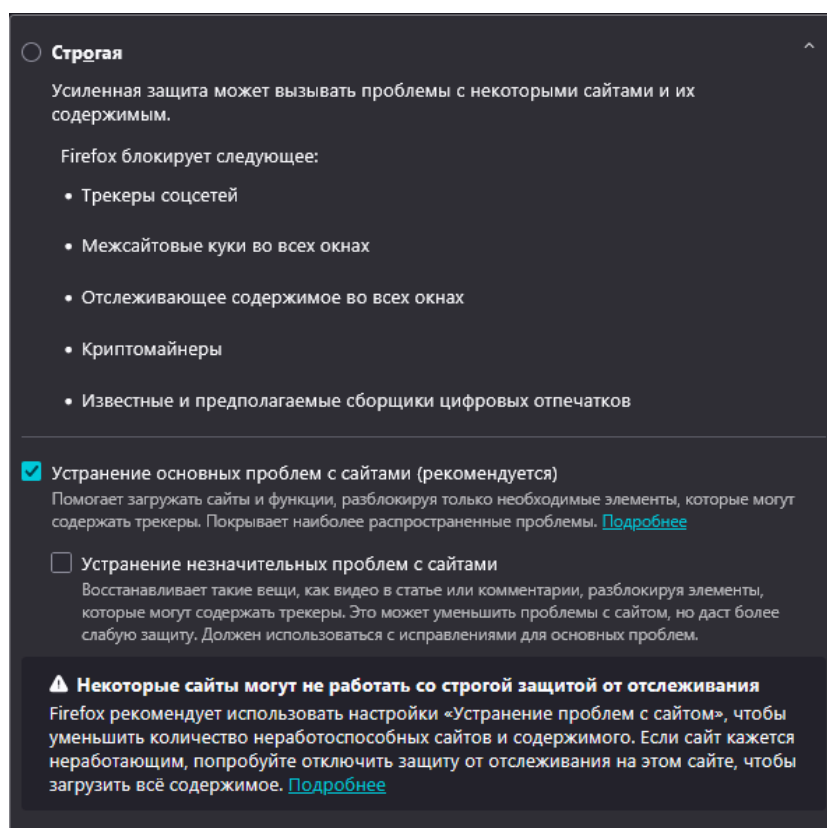


Рисунок 3 — Уровень настройки «Строгая»

Третья и последняя, *персональная*, как понятно по названию, даёт пользователю возможность настроить эти параметры по желанию — является более гибким по сравнению с прошлыми (рисунок 4).

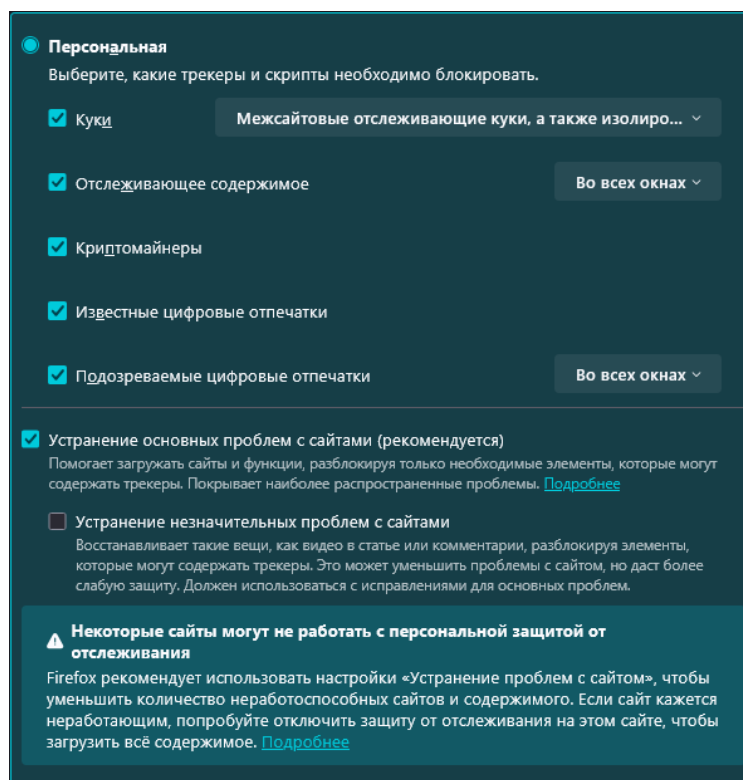


Рисунок 4 — Уровень настройки «Персональная»

Кроме этих, мною также был рассмотрен так называемый «белый список» исключений, который позволяет предоставить доступ к сторонним файлам куки для конкретных доменов несмотря на установленных запретов. Как показано на рисунке 5, список оставлен пустым в целях безопасности.

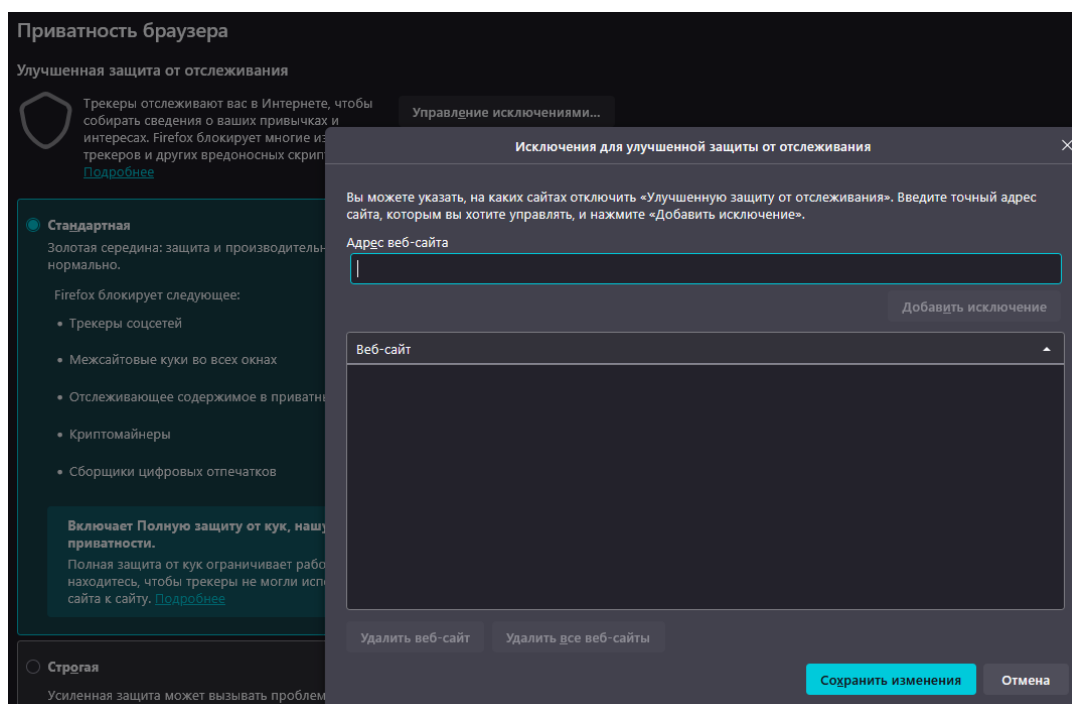


Рисунок 5 — «Белый список»

К сожалению, по сравнению с Chrome, в Firefox нет очевидных настроек на приватности рекламы. Поэтому я решил установить плагин, который косвенно выполняет аналогичную функцию (основное — блокировка рекламы). На рисунке 6 показан этот плагин.



Рисунок 6 — Плагин «Adblock Plus»

В конце я выбрал стандартный уровень защиты, так как оно является проверенным и надежным комплексом мер безопасности без жертвования функциональностей веб-сайтов. Этот уровень защиты функционирует против стандартных атак по типу фишинга и доступа к вредоносным ресурсам, тем самым обеспечивая необходимый уровень безопасности. В рисунках 7 и 8 показаны дополнительные параметры, сопровождающие этим требованиям.

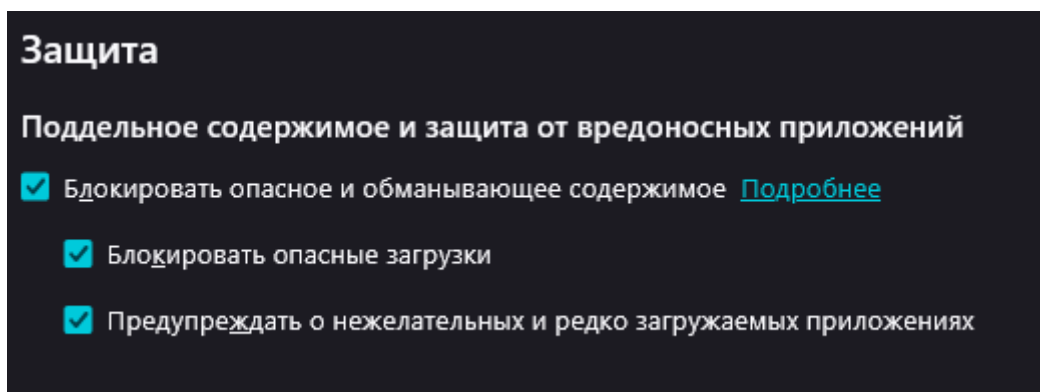


Рисунок 7 — Параметр защиты от вредоносных приложений

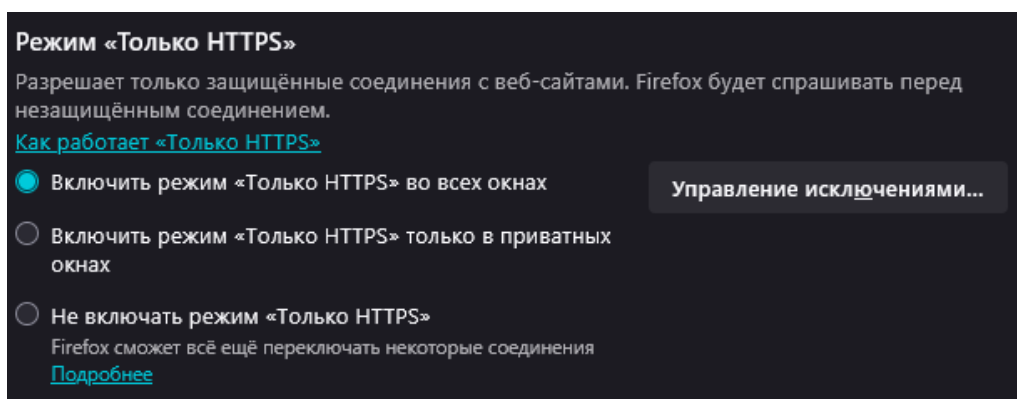


Рисунок 8 — Параметр защиты от незащищенных соединений

Также была отключена автосохранения и автозаполнения данных для аутентификации — это создает условие единой точки отказа, а также не защищает от использования лицами физическим образом.

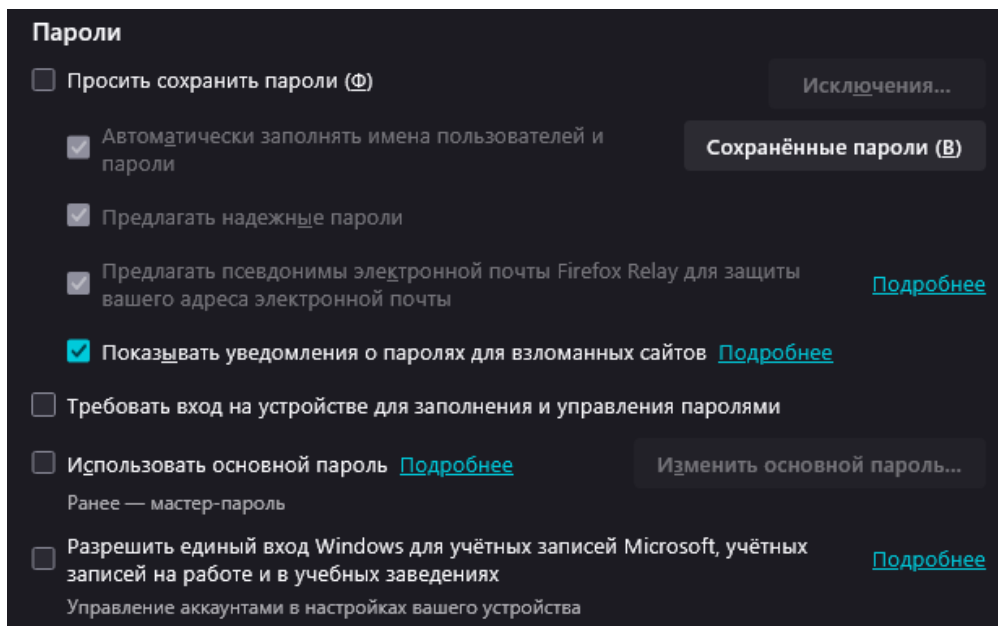


Рисунок 9 — Параметры о автозаполнения и сохранения паролей

Если теперь посмотреть раздел защиты от нежелательного выполнения JavaScript, то, как и с рекламами, у Firefox нет очевидной настройки для этого. Но этого можно обойти путем использования плагин, где в этот раз его главная функциональность является для меня целевым. На рисунке 10 показан этот плагин.



Рисунок 10 — Плагин «NoScript Security Suite»

Важно отметить, что эти плагины являются гибкими с точки зрения конфигурации, но для наших целей они являются достаточными даже если они были «из-за коробки».

Переходим в раздел сертификатов. Там можно будет смотреть и управлять сертификатами. На рисунке 11 видны доступные настройки сертификатов, а на 12 — сами сертификаты. Они играют центральную роль при установлении зашифрованного соединения TLSv1.3.

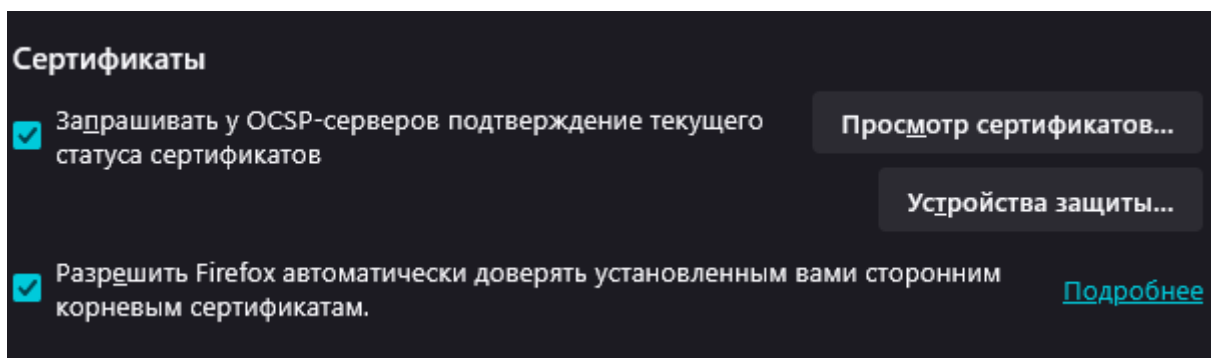


Рисунок 11 — Параметры управления сертификатами

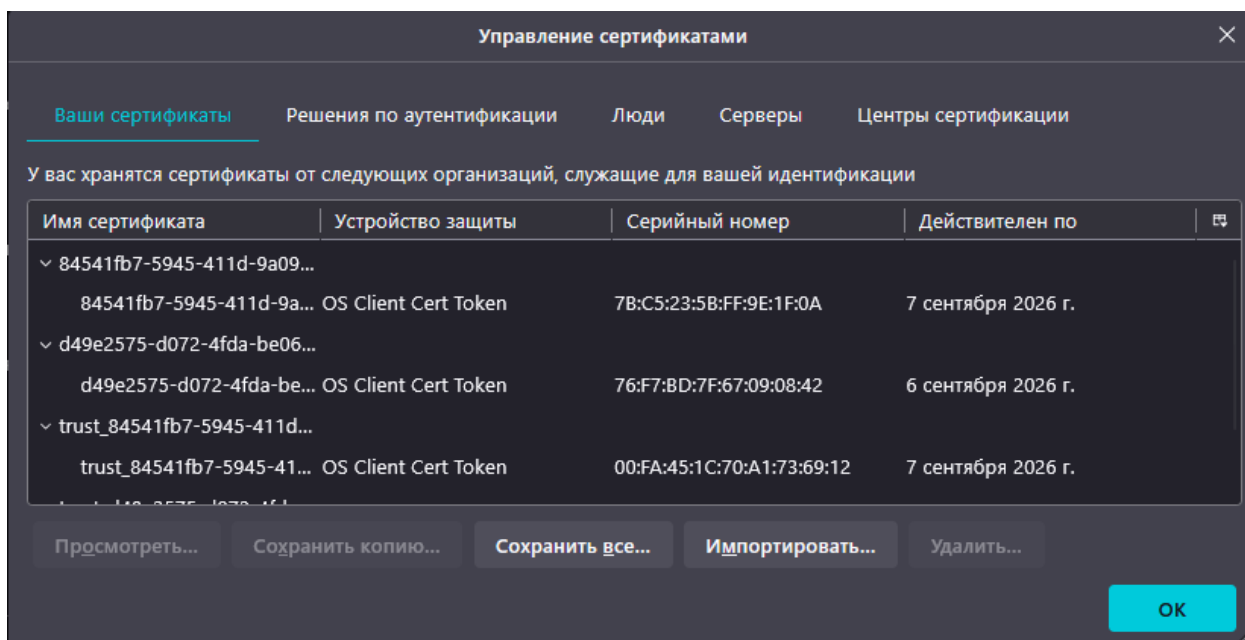


Рисунок 12 — Сертификаты на устройстве пользователя

Осталось проверить политику ограничения разрешения. Firefox обычно заставит сайтов спросить разрешение пользователя на первом посещении, но после этого сайту разрешено использовать одну и ту же настройку, что нежелательно в условиях безопасности.

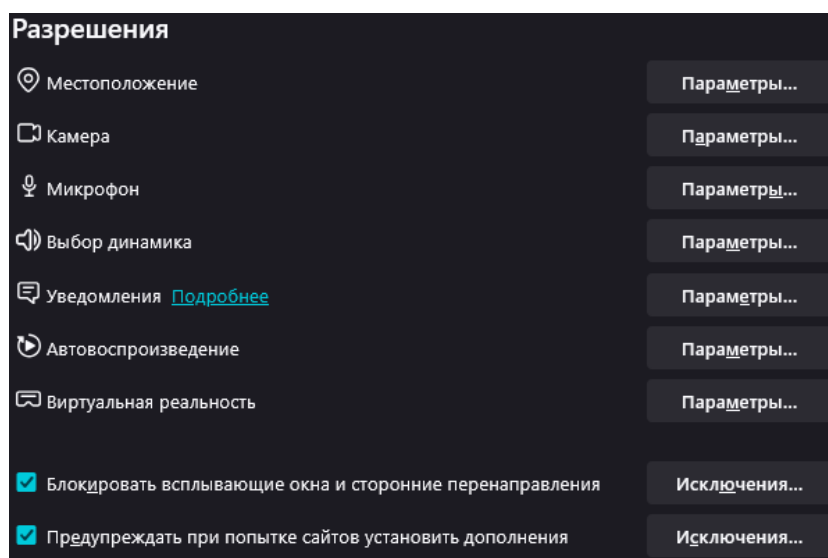


Рисунок 13 — Настройки разрешения

В таком случае лучше настроить разрешения устройства так, чтобы они спрашивали пользователя каждый раз при его использовании. В рисунке 14 показан, как это работало на примерном сайте, а в 15 — подтверждение работоспособности настройки.

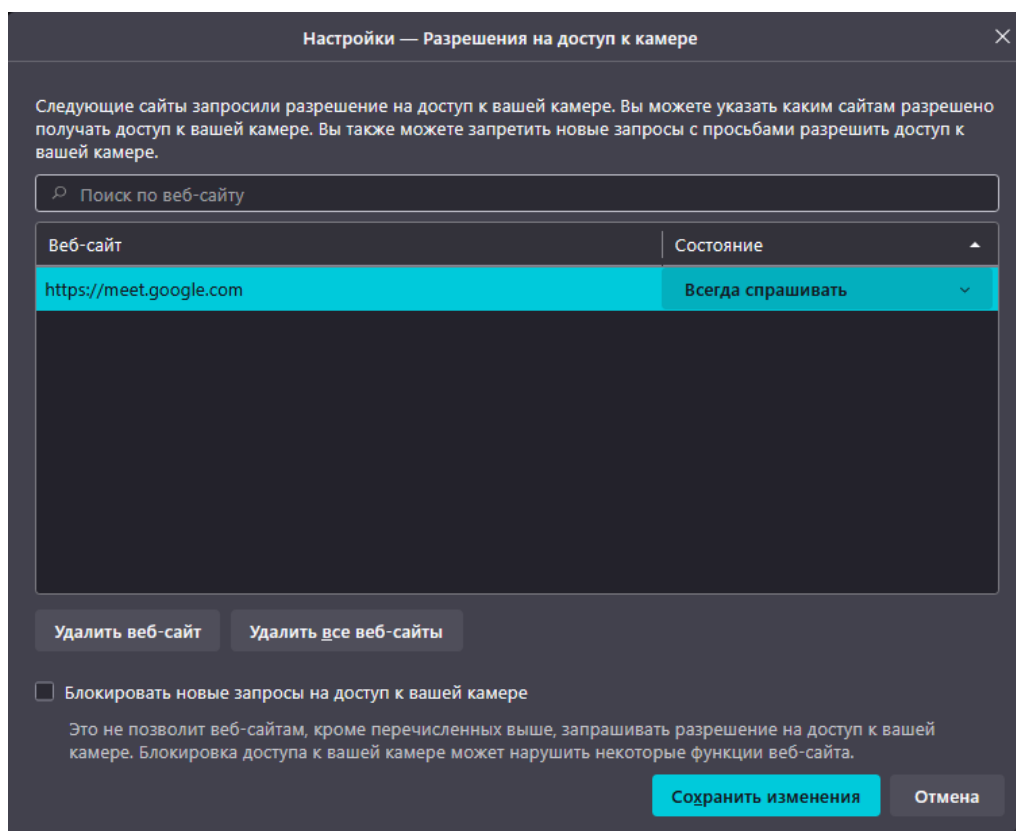


Рисунок 14 — Настройка разрешения использования камеры (пример)

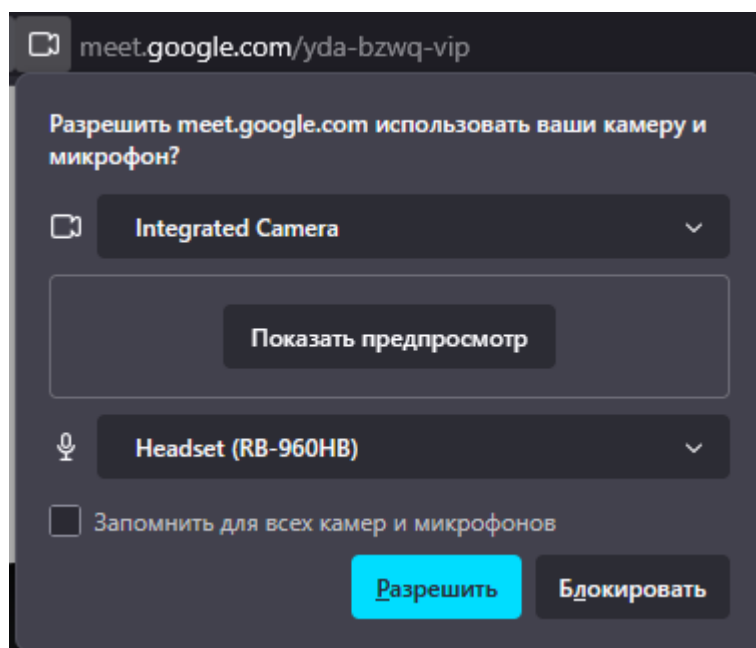


Рисунок 15 — Запрос от веб-сайта к пользователю с просьбой разрешить использования устройств

Чтобы закончить настройку, придется очистить кэш и файлы куки. На рисунке 16 показаны дополнительные настройки, связанные с удалением.

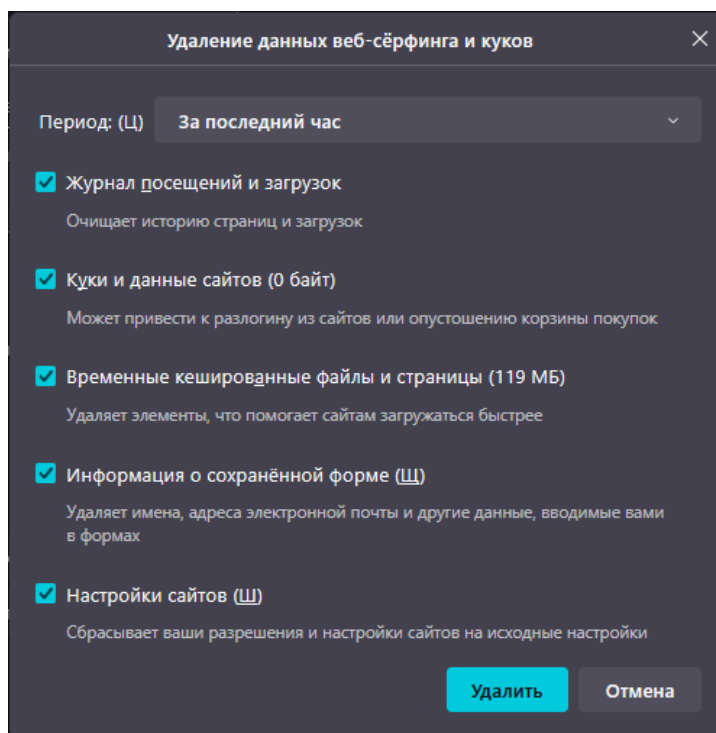


Рисунок 16 — Параметры очистки кэша и файлов куки
После очистки освободился примерно 119 Мб пространства.

Анализ трафика

Заходим на сайт <https://w3schools.com/>. Нажмём на F12 и перейдем во вкладку «Сеть». На картинке 17 есть весь вид вкладки:

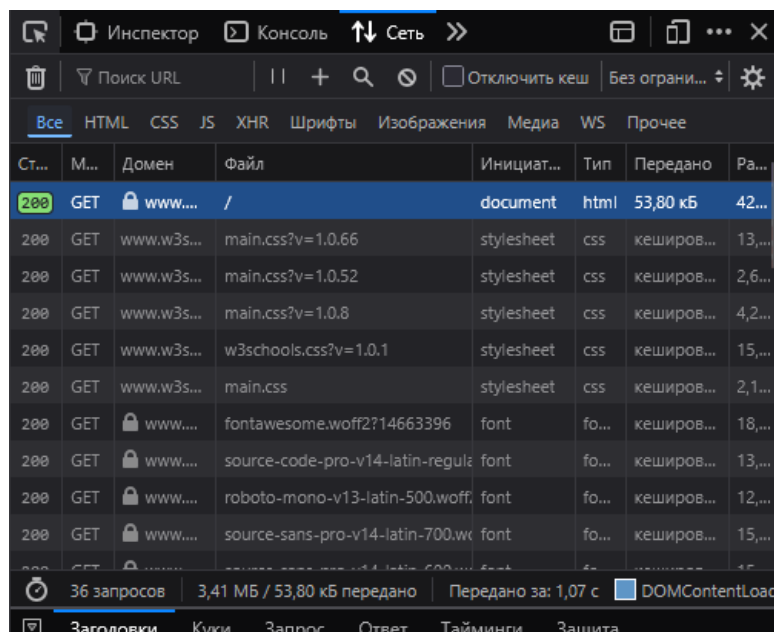


Рисунок 17 — Вкладка «Сеть»

Видно, что при обычной загрузке страницы было отправлено на сервер 36 HTTP-запросов. Если зайти в заголовках выделенного запроса:

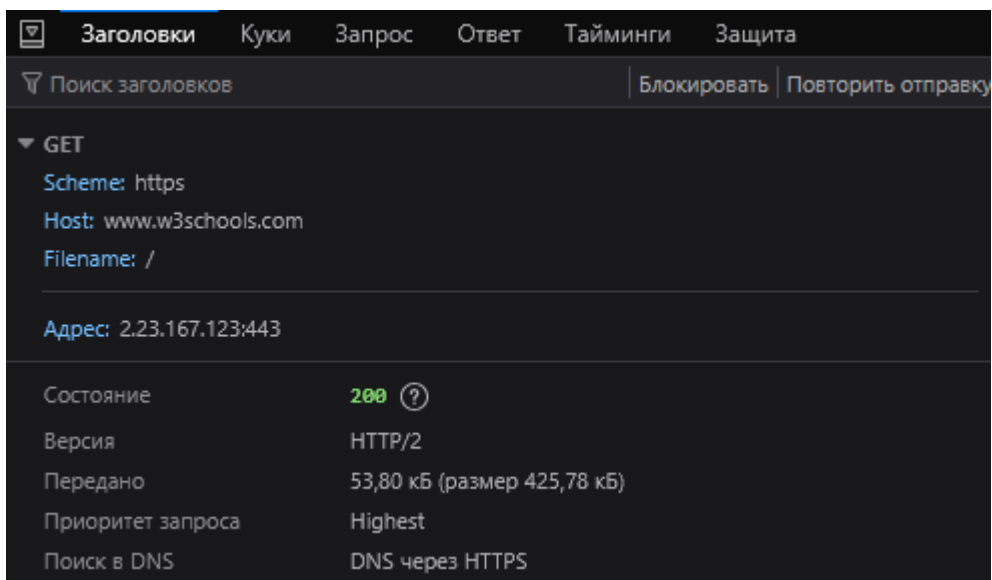


Рисунок 18 — Общие сведения запроса GET

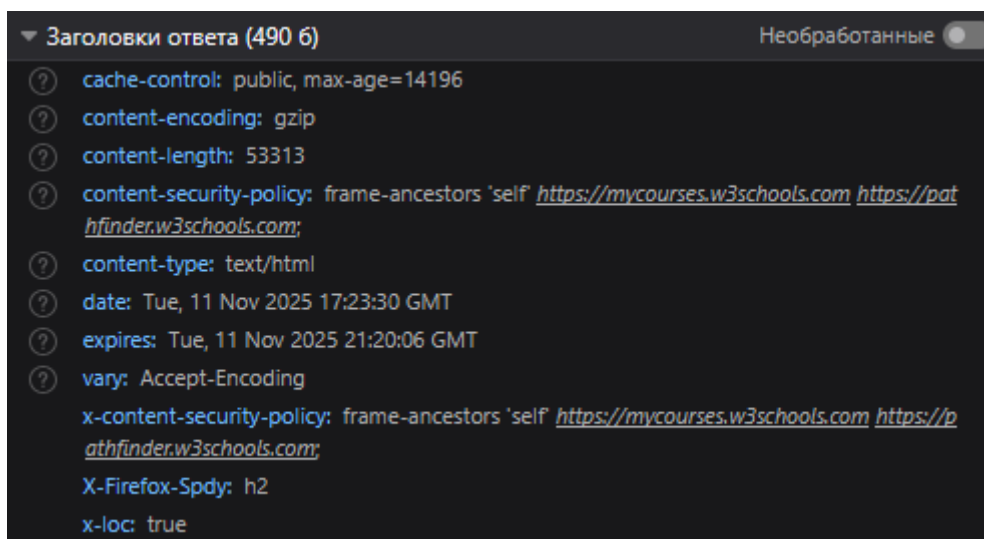


Рисунок 19 — Заголовки ответа GET

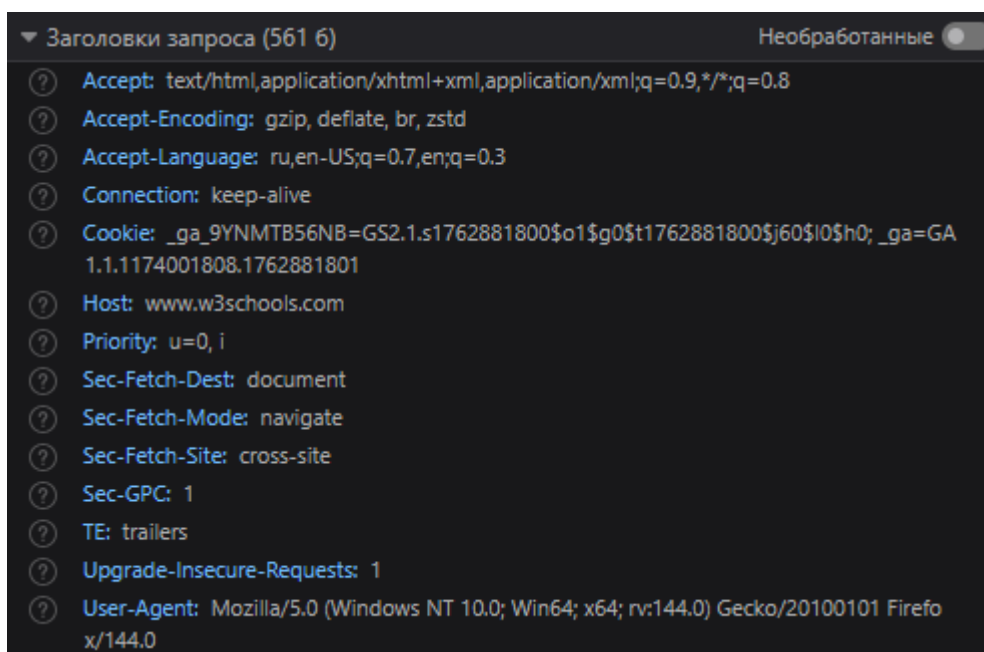


Рисунок 20 — Заголовки запроса GET

Разбираемся со всеми записями заголовков:

- Заголовки ответа:
 - cache-control: public, max-age: 14196 — разрешающая политика кэширования: контент может храниться в публичных кэшах, максимальный срок актуальности составляет 14196 секунд (около 3.94 часа) без обращения к серверу.
 - content-encoding: gzip — контент передан в сжатом формате для уменьшения объема передаваемых данных и ускорения загрузки.
 - content-length: 53313 — точный размер тела ответа в байтах (53313 байт), используется браузером для определения прогресса загрузки.

- content-security-policy: frame-ancestors 'self'
<https://mycourses.w3schools.com> <https://pathfinder.w3schools.com>;
 — политика безопасности, ограничивающая встраивание страницы во фреймы только с того же источника и указанных доверенных доменов для защиты от кликджекинга.
- content-type: html — тип контента определен как HTML, однако отсутствует указание кодировки и полного MIME-типа (стандартный формат должен быть text/html).
- date: Tue, 11 Nov 2025 17:23:30 GMT — точная дата и время формирования ответа сервером в формате GMT.
- expires: Tue, 11 Nov 2025 21:20:06 GMT — дата и время истечения срока действия контента, после которого кэш должен запросить обновленную версию с сервера.
- x-content-security-policy: frame-ancestors 'self'
<https://mycourses.w3schools.com> <https://pathfinder.w3schools.com>;
 — устаревшая версия заголовка CSP для поддержки старых браузеров (особенно Internet Explorer), дублирующая основную политику безопасности.
- X-Firefox-spdy: h2 — специфичный для Firefox заголовок, указывающий на использование протокола HTTP/2 для данного соединения.
- x-loc: true — нестандартный внутренний заголовок приложения, вероятно, указывающий на географическое расположение или внутреннюю логику маршрутизации сервера.
- Заголовки запроса:
 - Идентификации браузера и системы:
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:144.0) Gecko/20100101 Firefox/144.0 — идентификация клиента: браузер Firefox версии 144.0, работающий на 64-битной Windows 10. Содержит детальную информацию о платформе и движке Gecko, что может использоваться для таргетирования контента или анализа уязвимостей.
 - Host: www.w3schools.com — указание целевого домена для виртуального хостинга, позволяет серверу определить, какой сайт запрашивается при использовании одного IP-адреса для нескольких доменов.
 - Accept-Language: ru, en-US;q=0.7,en;q=0.3 — предпочтения языка пользователя: основной язык – русский, затем американский английский с приоритетом 0.7, и общий английский с приоритетом 0.3 для локализации контента.
 - Заголовки управления запросом и кешированием:
 - Accept: text/html,application/xhtml+xml,application/xml;q=0.9,

`*/*;q=0.8` — указание предпочитаемых MIME-типов: приоритет отдается HTML-контенту, затем XHTML и XML с коэффициентом 0.9, остальные типы принимаются с коэффициентом 0.8.

- `Accept-Encoding: gzip, deflate, br, zstd` — поддерживаемые методы сжатия данных: `gzip`, `deflate`, `Brotli (br)` и `Zstandard (zstd)`, что позволяет серверу выбрать оптимальный алгоритм для уменьшения объема передаваемых данных.
- `Connection: keep-alive` — указание поддерживать постоянное соединение с сервером для нескольких запросов, что повышает производительность и снижает накладные расходы на установление новых соединений.
- `TE: trailers` — запрос поддержки передачи дополнительных заголовков (трейлеров) в конце сообщения при использовании чанкового кодирования, что позволяет передавать метаданные после основного контента.
- `Priority: u=0, i` — указание приоритета загрузки ресурсов: `u=0` означает высокий приоритет для основного документа, `i` указывает на немедленную загрузку, что оптимизирует отображение страницы.

○ Заголовки безопасности и приватности:

- `Sec-Fetch-Dest: document` — указание назначения запрашиваемого ресурса как HTML-документа верхнего уровня, что помогает серверу принимать решения о безопасности и политике CORS.
- `Sec-Fetch-Mode: navigate` — указание режима запроса как навигации (переход на новую страницу), что отличает его от встроенных ресурсов или API-вызовов.
- `Sec-Fetch-Site: cross-site` — указание, что запрос инициирован с другого сайта (кросс-сайтовый), что влияет на политики безопасности и ограничения для межсайтовых запросов.
- `Sec-GPC: 1` — включение Global Privacy Control (GPC): сигнал о предпочтении пользователя не отслеживаться, что обязывает сайты уважать это требование в соответствии с законодательством о приватности.
- `Upgrade-Insecure-Requests: 1` — запрос автоматического обновления небезопасных HTTP-ресурсов до HTTPS, что улучшает безопасность и предотвращает смешанное содержимое на странице.

○ Работа с сессиями и аутентификацией:

- `Cookie:`
`_ga_9YNMTB56NB=GS2.1.a1762881800$o1$g0$t176288180`

0\$j60\$I0\$h0; _ga=GA1.1.1174001808.1762881801 — отсутствие куки в запросе указывает на отсутствие активной сессии или очистку куки, что может быть связано с приватным режимом браузера или настройками безопасности пользователя.

Если рассматривать POST-запрос (рисунок 21):

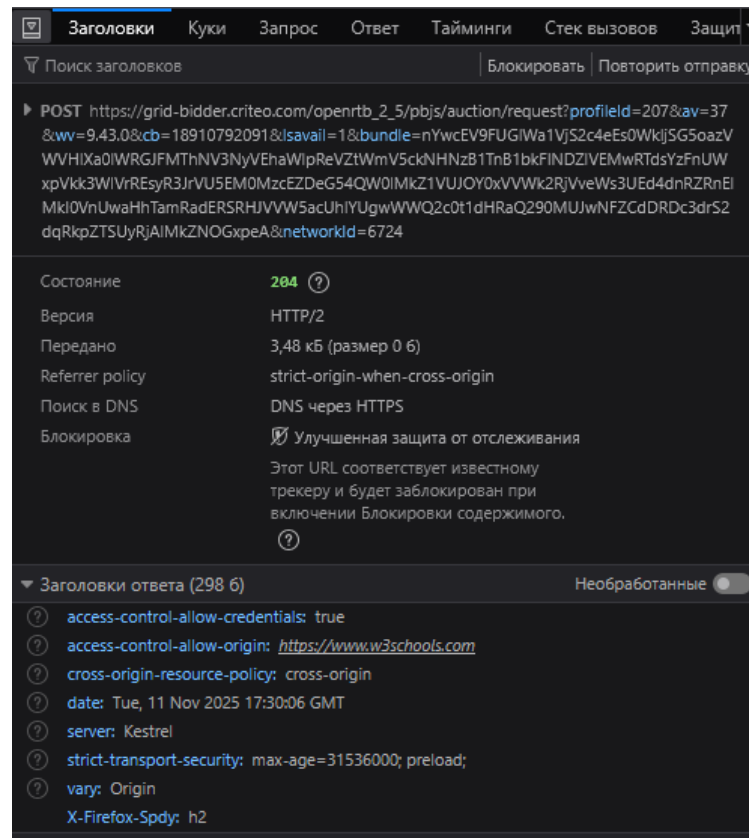


Рисунок 21 — Вкладка «Сеть» при POST-запросе

Те же общие заголовки. Если рассматривать заголовков ответа (рисунок 22) и запроса (рисунок 23)

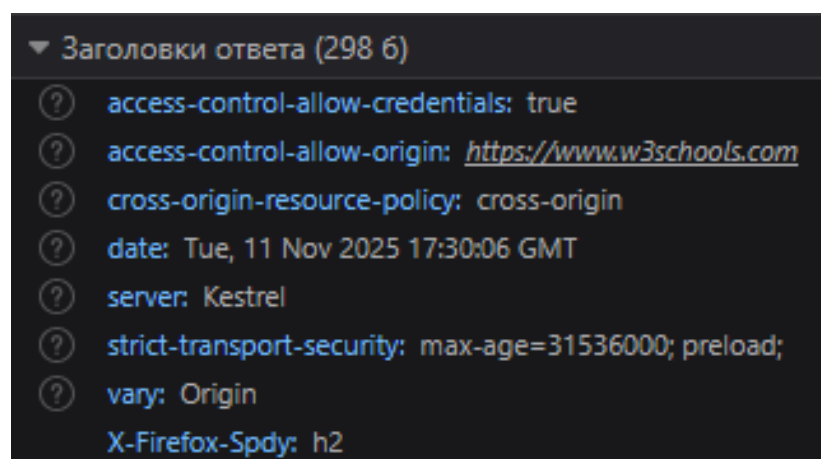


Рисунок 22 — Заголовки ответа POST-запроса

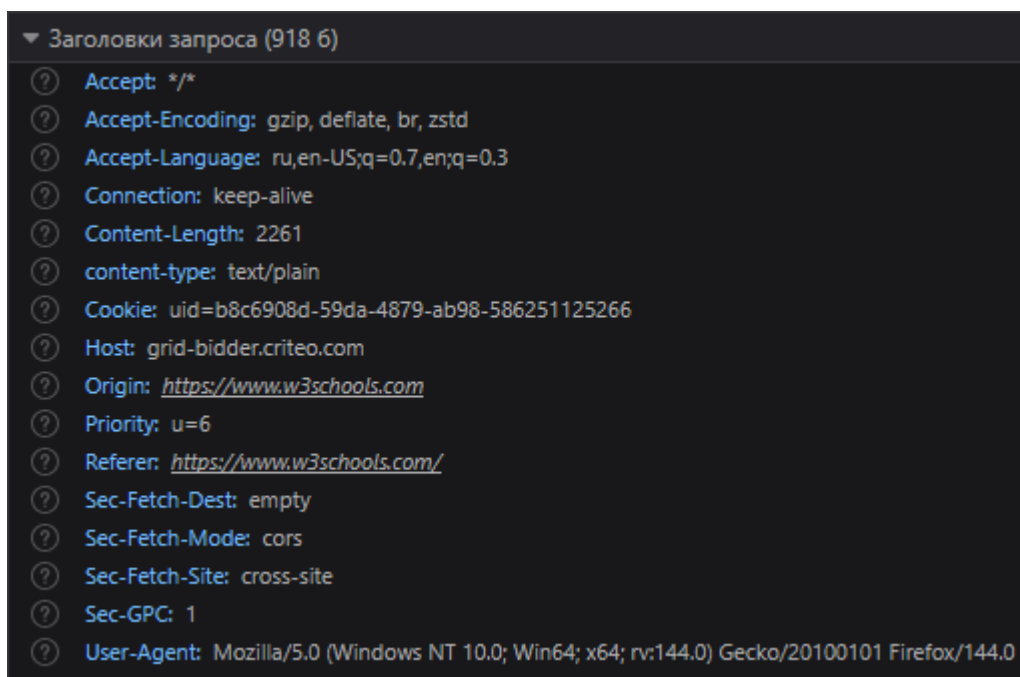


Рисунок 23 — Заголовки запроса POST-запроса

Разбираемся со всеми записями заголовков:

- Заголовки ответа:
 - access-control-allow-credentials: true — разрешение передачи учетных данных (куки, заголовков авторизации) в кросс-доменных запросах, что позволяет аутентифицированным пользователям взаимодействовать с ресурсом из другого источника.
 - access-control-allow-origin: <https://w3schools.com> — политика CORS, разрешающая доступ к ресурсу только с указанного домена w3schools.com, что ограничивает кросс-доменные запросы только доверенным источником.
 - cross-origin-resource-policy: cross-origin — разрешение загрузки ресурса с любого источника, что позволяет другим сайтам встраивать или запрашивать данный ресурс без ограничений.
 - date: Tue, 11 Nov 2025 17:30:06 GMT — точное время формирования ответа сервером в формате GMT, используемое для синхронизации кэширования и отладки временных проблем.
 - server: Kestrel — идентификация веб-сервера как Kestrel (легковесный кросс-платформенный сервер для ASP.NET Core), раскрытие версии которого может представлять угрозу безопасности при наличии известных уязвимостей.
 - strict-transport-security: max-age=31536000; preload; — политика HSTS (HTTP Strict Transport Security), принудительно требующая использование HTTPS в течение 1 года (31536000 секунд) и указывающая на готовность сайта к включению в

предварительные списки HSTS браузеров для максимальной защиты от downgrade-атак.

- vary: Origin — инструкция для кэшей о том, что ответ зависит от заголовка Origin в запросе, что гарантирует правильное кэширование разных версий ресурса для разных источников.
- X-Firefox-Spdy: h2 — специфичный для Firefox заголовок, указывающий на использование протокола HTTP/2 (ранее SPDY) для данного соединения, что обеспечивает улучшенную производительность и мультиплексирование запросов.
- Заголовки запроса:
 - Идентификации браузера и системы:
 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:144.0) Gecko/20100101 Firefox/144.0 — детальная идентификация клиента: браузер Firefox версии 144.0 на базе движка Gecko, работающий на 64-битной Windows 10. Содержит полную информацию о платформе и версии браузера, что используется серверами для контент-неготиации и анализа совместимости.
 - Host: grid-bidder.criteo.com — указание целевого домена сервера для виртуального хостинга, позволяющее обрабатывать несколько сайтов на одном IP-адресе. В данном случае запрос направлен к рекламному сервису Criteo для аукциона рекламных мест.
 - Accept-Language: ru, en-US;q=0.7,en;q=0.3 — предпочтения языков пользователя: основной язык – русский (без коэффициента), американский английский с приоритетом 0.7, общий английский с приоритетом 0.3. Используется сервером для локализации контента и интерфейса.
 - Заголовки управления запросом и кэшированием:
 - Accept-Encoding: gzip, deflate, br, zstd — поддерживаемые методы сжатия данных: gzip (стандартный), deflate (устаревший), Brotli (br) и Zstandard (zstd). Позволяет серверу выбрать наиболее эффективный алгоритм для уменьшения объема передаваемых данных и ускорения загрузки.
 - Connection: keep-alive — указание поддерживать постоянное TCP-соединение для нескольких запросов, что снижает накладные расходы на установление новых соединений и повышает производительность взаимодействия с сервером.
 - Priority: u=6 — указание приоритета загрузки ресурса, где u=6 означает низкий приоритет (фоновая загрузка). Используется браузером для оптимизации порядка

загрузки ресурсов в зависимости от их важности для отображения страницы.

○ Заголовки безопасности и приватности:

- Sec-Fetch-Dest: empty — указание, что запрашиваемый ресурс не имеет конкретного назначения или используется для служебных целей (например, предварительные запросы, пинг-запросы). Отличается от стандартных значений вроде document, script или image.
- Sec-Fetch-Mode: cors — указание режима запроса как CORS (Cross-Origin Resource Sharing), что означает кросс-доменный запрос с соблюдением политик безопасности. Требуем от сервера включения соответствующих заголовков CORS для успешного выполнения.
- Sec-Fetch-Site: cross-site — указание, что запрос инициирован с другого сайта (кросс-сайтовый), что влияет на применение политик безопасности и ограничений для межсайтовых запросов, особенно в контексте защиты от CSRF-атак.
- Sec-GPC: 1 — включение Global Privacy Control (GPC): сигнал о предпочтении пользователя не подвергаться отслеживанию, что обязывает сайты и сервисы уважать это требование в соответствии с законодательством о защите данных и приватности.

○ Работа с сессиями и аутентификацией:

- Cookie: uid=b8c6908d-59da-4879-ab98-586251125266 — передача идентификатора пользователя (UUID) в качестве куки, используемого для отслеживания сессии, персонализации контента или аналитики. В данном случае уникальный идентификатор может использоваться для таргетирования рекламы.

○ Технические заголовки:

- Content-Length: 2261 — точный размер тела запроса в байтах (2261 байт), который помогает серверу определить конец данных и корректно обработать запрос без ожидания дополнительных данных.
- Accept: */* — указание, что клиент принимает любой MIME-тип контента. Используется в служебных запросах, где конкретный тип контента не важен, или когда клиент готов обработать любой возвращаемый сервером формат.

○ Заголовки источника запроса:

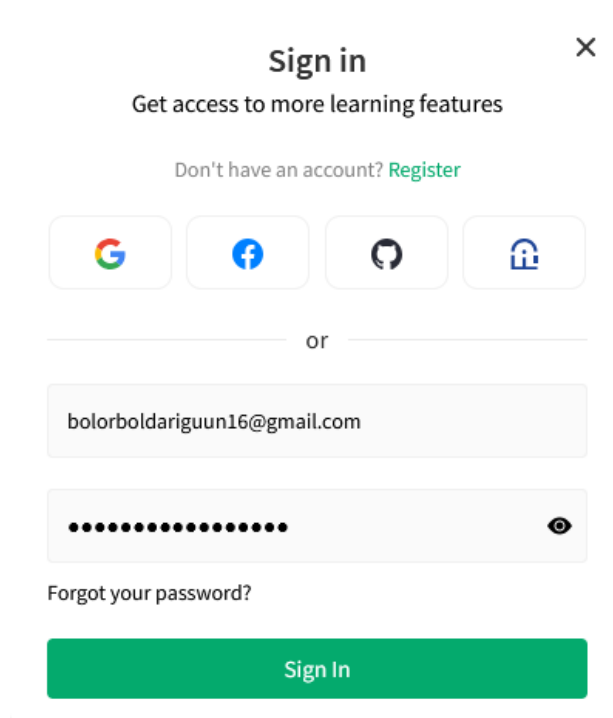
- Origin: <https://w3schools.com> — указание источника, инициировавшего кросс-доменный запрос. Требуется для CORS-запросов и позволяет серверу принимать решения о

разрешении или запрете доступа на основе политики безопасности.

- Referer: <https://w3schools.com/> — указание URL-адреса страницы, с которой был инициирован запрос. Используется для аналитики, контроля доступа и понимания контекста запроса, но может быть отключен в целях приватности.

Форма входа

Далее на этом же сайте введем тестовые данные в форму входа, как показано на рисунке 24:



Sign in

Get access to more learning features

Don't have an account? [Register](#)

or



[Forgot your password?](#)

[Sign In](#)

Рисунок 24 — форма входа

После ввода данных в форму появился POST-запрос (рисунок 25):

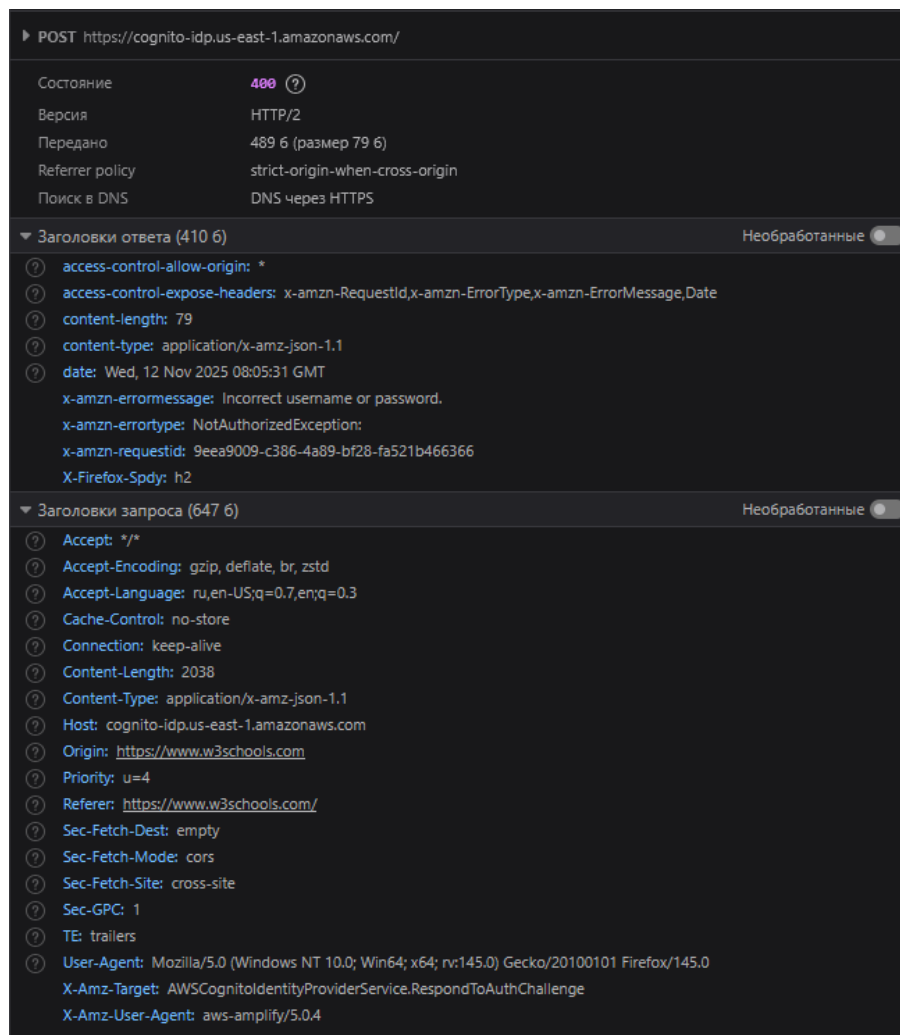


Рисунок 25 — POST-запрос с пользовательскими данными

Для проверки того, как выглядит запрос (точнее полезная нагрузка), откроем вкладку «Запрос» (рисунок 26):

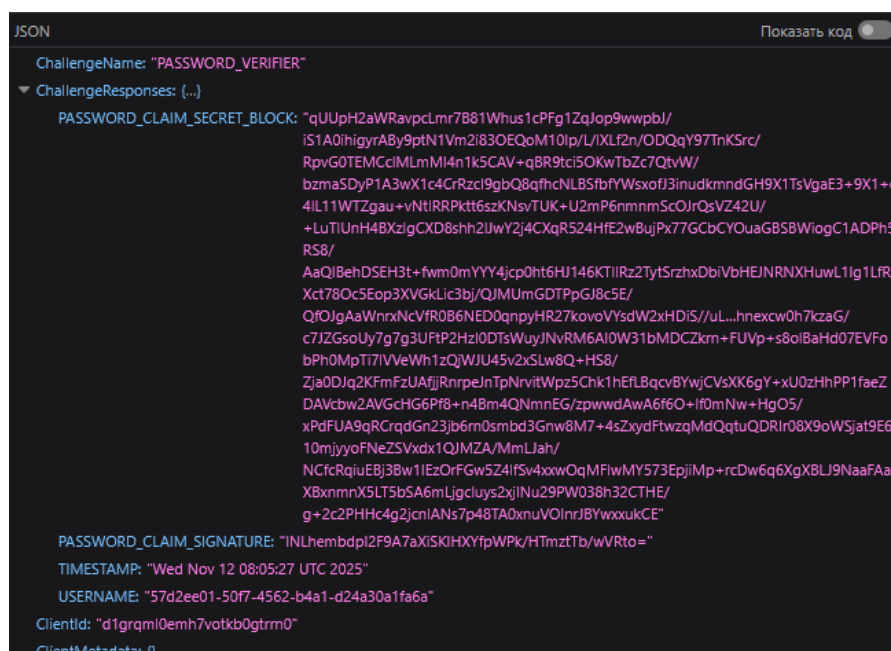


Рисунок 26 — Полезная нагрузка

Статус ответа: 400 Bad Request — указывает на то, что запрос был принят и обработан сервером, но сам процесс аутентификации закончился неуспешным (неправильные данные).

Ещё видно, что данные зашифрованы при передаче. Так как w3schools использует AWS, то следовало бы ожидать довольно адекватного уровня безопасности, так как на AWS опирается значительная доля всего инфраструктуры глобальной сети.

Как чаще всего передаются данные аутентификации (логин/пароль) в современных веб-приложениях?

В современных веб-приложениях данные аутентификации чаще всего передаются через защищенные HTTPS-соединения с использованием HTML-форм. Пользователь вводит логин и пароль в специальную форму на веб-странице, после чего эти данные отправляются на сервер методом POST в теле запроса. Важно, что такая передача осуществляется исключительно по протоколу HTTPS, который обеспечивает шифрование данных и защищает их от перехвата злоумышленниками в публичных сетях. Сервер затем проверяет учетные данные и, при успешной аутентификации, устанавливает сессию для пользователя.

После первоначальной аутентификации современные приложения обычно переключаются на использование токенов или сессионных идентификаторов для последующих запросов. Для веб-интерфейсов чаще применяются сессионные куки, которые автоматически прикрепляются браузером к каждому запросу. В API-ориентированных приложениях и одностраничных приложениях (SPA) распространена практика использования JWT (JSON Web Tokens) или других токенов, которые передаются в заголовке Authorization с префиксом “Bearer ”. Это позволяет избежать повторной передачи пароля при каждом запросе и улучшает безопасность.

Современные системы аутентификации также активно используют стандарты OAuth 2.0 и OpenID Connect для делегированной аутентификации через социальные сети или корпоративные системы единого входа. В таких случаях пароль никогда не передается напрямую основному приложению — вместо этого происходит перенаправление пользователя к провайдеру аутентификации, а приложению передается только авторизационный код или токен. Независимо от выбранного метода, обязательным требованием остается использование HTTPS для всех этапов аутентификации и передачи любых чувствительных данных.